

一文详解 GPT-Live 的技术架构与实现方案

从端到端 Omni Model 到全双工连续推理系统

Jiguo Li¹

jiguolee@gmail.com

2026 年 8 月

摘要

本文缘起于 OpenAI 近期发布的一篇 GPT-Live 工程博客。该文首次较系统地披露了这一新一代语音系统如何从轮次式架构转向连续交互，并介绍了 continuous inference、异步委派、有状态实例切换和低时延传输等关键设计，由此引发了笔者对其完整技术架构的兴趣。GPT-Live 将语音助手从“识别一句、思考一次、合成一句”的轮次管线，推进为始终在线的连续控制系统：模型沿统一时间轴持续接收用户音频，在发声的同时保持监听，判断何时暂停或接受打断，并将复杂推理和工具调用异步交给更强的模型。本文结合 OpenAI 官方工程说明、系统卡、Internet Engineering Task Force (IETF) 协议草案，以及 2024–2026 年全双工语音模型论文与开源项目，给出一份受公开证据约束的技术重建。核心结论是：GPT-Live 的创新不能只归结为“更强的端到端 speech-to-speech model”，而应理解为四层协同——全双工交互核心、异步认知委派、stateful serving 和低往返时延实时传输。本文进一步形式化 continuous inference，比较 codec token、codec-free、双流融合、cross-attention、finite-state machine control token 与 action channel 等公开路线；分析训练数据、长会话状态、attention cache、上下文压缩、故障切换和 WebRTC/WARP；最后给出可复现的开源实现蓝图、消融实验与多维评测体系。为避免把类比误写成内幕，全文使用 [F1] 官方事实、[F2] 公开机制和 [H] 推断性重建三级证据标签。

关键词：GPT-Live；全双工语音模型；continuous inference；speech-to-speech；实时系统；WebRTC；stateful serving

阅读提要：GPT-Live 的关键不只是一个端到端 speech-to-speech model，而是 full-duplex interaction core、异步 cognition、stateful serving 与低延迟实时传输的协同。

¹ 本文在 OpenAI Codex 的协助下完成。

目录

1	问题定义、证据边界与演进路线	3
1.1	研究缘起：从 OpenAI 的一篇工程博客说起	3
1.2	本文究竟解释什么	3
1.3	四代语音交互范式	3
1.4	GPT-Live 的官方系统轮廓	4
2	全双工连续推理的形式化	4
2.1	从 token 序列到受 wall-clock 约束的双流过程	4
2.2	状态、因果性与回声	5
2.3	双时间尺度控制	5
3	公开模型路线：组件与设计轴	5
3.1	音频表示：连续特征、semantic token 与 codec token	5
3.2	双方流如何进入主干	6
3.3	交互动作如何表达	6
3.4	与端到端 Omni Model 的本质区别	6
4	训练数据与后训练	6
4.1	为什么普通语音问答数据不够	6
4.2	建议的渐进训练配方	7
4.3	数据流水线的可量化质量控制	7
5	Stateful Serving：把 audio frame 按时送达	7
5.1	容量单位从 request 转变为 session-frame	7
5.2	KV cache 与连续上下文	8
5.3	无缝实例切换	8
5.4	异步委派的 serving 设计	8
6	实时网络：WebRTC、WARP 与 playout clock	9
6.1	为什么不是“换成 WebSocket 就够了”	9
6.2	从 6 RTT 到 1 RTT 的建连优化	9
6.3	丢包、漂移与音频追赶	9
7	评测体系：从模型分数到连续交互体验	10
7.1	四层评测框架	10
7.2	关键指标定义	10
7.3	实验矩阵与归因原则	10
8	证据约束下的 GPT-Live 架构重建	11
8.1	哪些部分可以确定	11
8.2	最可信但尚未证实的内部机制	11
8.3	“混合架构”比“纯端到端”更准确	11
9	一个可复现的开源实现方案	12
9.1	最小可行架构	12
9.2	帧级调度伪代码	12
9.3	分阶段实验计划	12
9.4	资源与延迟预算示例	13
10	局限性、开放问题与结论	13
10.1	仍然未知的关键细节	13
10.2	研究前沿	13
10.3	结论	13

1 问题定义、证据边界与演进路线

1.1 研究缘起：从 OpenAI 的一篇工程博客说起

本文的直接缘起，是 OpenAI 于 2026 年 8 月 3 日发布的工程博客《How We Built a Realtime System for Responsive Voice AI in Six Months》[1]。与通常侧重功能展示和评测结果的产品介绍不同，这篇文章从工程实现出发，解释了团队为何放弃依赖轮次检测器的传统语音架构，以及如何在六个月内重新设计推理、上下文管理和媒体传输系统。博客披露的若干细节尤其引起了笔者的兴趣：语音模型可以同时听和说；媒体帧通过独立的快速路径传输；复杂推理和工具调用沿另一条异步路径执行；长会话通过后台上下文压缩和模型实例切换保持连续；WARP 与 Instant Connect 则试图进一步缩短 WebRTC² 建连时间。

这些公开信息勾勒出了 GPT-Live 的系统轮廓，却也留下了更值得追问的问题：全双工语音模型如何在生成期间持续接收新输入？模型如何表示双方重叠的语音以及暂停、简短应答和打断等交互动作？快速交互模型与深层推理模型之间如何传递上下文和结果？一个持续数十分钟的有状态会话，又如何在 GPU 调度、KV 缓存、网络抖动和实例故障之下保持流畅？

这篇博客因此成为本文的出发点。本文并不复述其产品功能，而是以其中公开的系统边界为线索，结合近几年的顶会论文、技术报告、开源实现和网络协议，尝试还原一套技术上自洽、证据上可追溯的 GPT-Live 架构图景。换言之，本文要回答的不只是“GPT-Live 有哪些能力”，而是“为了实现这些能力，模型与系统分别需要解决什么问题”。

1.2 本文究竟解释什么

“GPT-Live 如何实现”包含两个容易混淆的问题。其一是模型问题：音频如何表示，用户流如何进入 Transformer，模型如何同时预测语义、声学与交互动作。其二是系统问题：网络抖动、连续会话、GPU 推理时限、KV 缓存、复杂推理与工具调用如何在不中断语音交互的条件下协同工作。GPT-Live 的公开材料表明，系统问题已经与模型问题同等重要 [1, 2]。因此，本文研究的是“能够处理自然语音重叠、随时接受打断、持续反馈并执行后台任务的在线语音智能体”，而不是某个孤立的模型检查点。

证据采用三级标注：**[F1]**仅来自 OpenAI 官方文章和系统卡；**[F2]**来自同行评审论文、作者技术报告、开源代码或标准；**[H]**是由多项事实共同约束的架构重建。尤其需要强调，OpenAI 没有公开 GPT-Live 的参数量、音频 tokenizer、frame rate、attention 连接方式、训练 token 数或集群规模。Moshi、DuplexOmni 等工作可以说明一种机制在工程上可行，却不能证明 GPT-Live 使用同一机制。

1.3 四代语音交互范式

级联式语音管线。传统系统串联 VAD/endpointing、ASR、文本 LLM 和 TTS³。其优势是模块可观测、组件易替换、文本工具生态成熟；问题是误差逐层累积，韵律与非语言信号在文本瓶颈处丢失，而且每轮必须等待“用户说完”。总延迟可粗略写为

$$T_{\text{cascade}} = T_{\text{endpoint}} + T_{\text{ASR}} + T_{\text{LLM,first}} + T_{\text{TTS,first}} + T_{\text{net}}. \quad (1)$$

其中，端点检测（endpointing）往往是尾部延迟的主要来源。即便所有模型都支持流式处理，只要状态机仍强制各模块串行运行，交互本质上仍是半双工。

轮次式端到端 **Omni Model**。GPT-4o 将文本、视觉和音频统一到一个神经网络中，官方报告其音频响应最低可达 232 ms、平均为 320 ms [3]。**[F1]**这种架构缓解了级联管线中的模态信息损失，并降低了部分首包延迟，但“端到端”并不自然等同于“全双工”：如果推理 API⁴ 在生成期间不再接收新的用户音频帧，或仍依赖外部 VAD 判定一轮对话结束，系统依然是轮次式的。Qwen2.5-Omni 的 Thinker-Talker、GLM-4-Voice 与 Mini-Omni 展示了流式语音到语音的不同实现 [4-6]，但其交互调度能力仍需单独评估。

单模型全双工。Moshi、SyncLLM、LSLM 和 Neural-FSM 将时间以及用户、助手两条音频流显式纳入建模：同一时刻既有用户通道，也有助手通道，模型可以选择静默、给出简短应答、继续发言、停止或接受打断 [7-10]。此时，inference unit 从“一轮对话”缩短为固定时长的 audio frame 或 chunk；延迟关注点也从 time-to-first-token (TTFT)，转变为每个 audio frame 能否在 playout deadline 前完成。

交互-认知解耦的连续智能体。**[F1]**GPT-Live 将快速交互与复杂思考拆开：交互模型持续监听和发声，深层推理或工具调用异步委派给更强模型；结果准备好后再自然并入对话 [1, 2]。2026 年 DuplexOmni 独立提出 interaction

²Web Real-Time Communication, Web 实时通信技术栈，用于浏览器或客户端之间的低延迟音视频与数据传输。

³VAD: Voice Activity Detection, 语音活动检测; ASR: Automatic Speech Recognition, 自动语音识别; LLM: Large Language Model, 大语言模型; TTS: Text-to-Speech, 文本转语音。

⁴Application Programming Interface, 应用程序编程接口。

layer 与 pluggable thinking layer 的并行结构 [11]，为这种范式提供公开学术参照。它不是回到 ASR-LLM-TTS 级联，而是让低延迟闭环与高智能闭环工作在不同时间尺度。

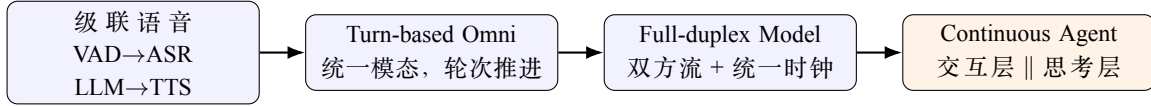


图 1: 语音模型的关键演进不是单纯扩大参数，而是逐步解除文本瓶颈、轮次屏障和单时间尺度推理。

1.4 GPT-Live 的官方系统轮廓

[F1]OpenAI 描述的线上系统可归纳为四个平面。第一，媒体平面 (**media plane**) 通过 WebRTC 持续收发音频，并处理 packet loss、jitter、clock drift 和 audio catch-up。第二，交互平面 (**interaction plane**) 运行 GPT-Live 语音模型，按照 audio clock 持续决定何时说、听、停、插话、调用工具或发起委派。第三，认知平面 (**cognition plane**) 维护预先创建和填充、具有稳定 session affinity 的 frontier-model session，用于承接复杂推理。第四，控制与状态平面 (**control/state plane**) 维护 session state、transcript、context compaction、安全控制和 instance handoff[1, 12]。

在媒体快速路径与应用逻辑之间设置异步 RPC⁵ 边界至关重要：数据库访问、业务逻辑或耗时工具都不能反向阻塞音频帧。系统因而存在两个一致性层级：音频路径采用“时限优先”的弱同步方式，应用状态则采用可重试的事件语义。连续音频之上仍可推导出离散的语义轮次，并同时维护临时转写 (speculative transcript) 和最终转写 (authoritative transcript)。因此，“取消轮次检测器”指的是取消会阻塞音频路径的轮次门控，而不是放弃语义分段。

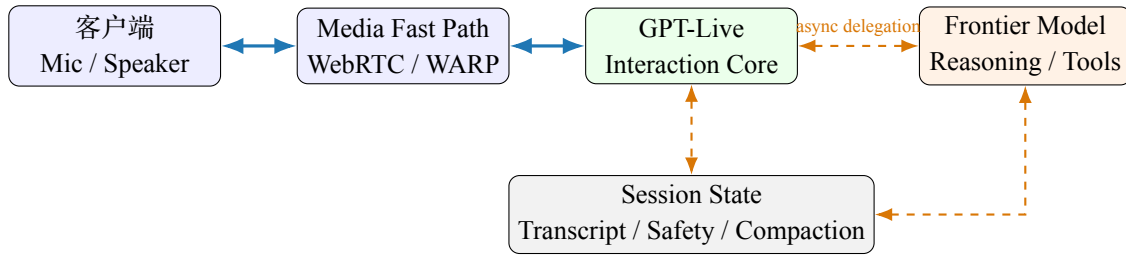


图 2: 证据约束下的 GPT-Live 系统轮廓。蓝色实线是低延迟媒体闭环，橙色虚线是允许更高延迟的认知与状态闭环。图中逻辑分层来自官方事实；模块内部实现仍未公开。

2 全双工连续推理的形式化

轮次式 LLM 只需保证 token 序列上的因果性；full-duplex model 还必须保证每个 audio chunk 的排队与计算时间满足 $Q_t + C_t \leq \Delta$ ，否则即使语义正确，也会因错过 playout deadline 而产生卡顿。因而 continuous inference 的状态不仅包含历史语义，还必须同步用户音频、助手自身输出、播放进度、工具事件和安全信号，并让低延迟 interaction path 与高成本 cognition path 在不同时间尺度上并行推进。

2.1 从 token 序列到受 wall-clock 约束的双流过程

下述形式化以标准 Transformer 自回归建模为起点 [13]。

普通自回归 LLM 建模 $p(y_n | x, y_{<n})$ ，输入 x 在解码开始前已经固定。全双工语音则不同：助手生成语音时，新的用户输入仍在持续到达。将 wall-clock 离散为长度为 Δ 的 audio chunk，令 u_t 表示第 t 个用户 audio chunk， a_t 表示助手输出， z_t 表示内部交互动作， m_t 表示持久状态，则系统需要在每个 playout deadline 之前完成以下计算：

$$m_t = f_{\theta}(m_{t-1}, u_t, \tilde{a}_{t-d:t-1}, e_t), \quad (2)$$

$$(a_t, z_t) \sim p_{\theta}(\cdot | m_t), \quad z_t \in \{\text{listen, speak, pause, stop, backchannel, delegate, tool}\}. \quad (3)$$

$\tilde{a}$ 表示系统自身的回放参考，或经过声学回声消除后的自身输出； e_t 表示工具结果、网络状态和安全信号。这里最重要的约束不是概率公式本身，而是 $C_t + Q_t \leq \Delta$ ：单个 audio chunk 的计算时间 C_t 与排队时间 Q_t 之和必须小于 playout deadline。偶发超时会造成 audio underrun，连续超时则会表现为机械式停顿。

全双工至少包含四层含义：（1）传输层可同时上行和下行；（2）模型在发声时仍持续编码用户输入；（3）策略层能够处理语音重叠、简短应答和用户插话；（4）会话层在后台任务执行期间仍保持可交互。仅具备双向 WebSocket 连接并不能称为全双工模型；如果系统在处理复杂任务时冻结交互，即使具备前三层能力，也不等于 GPT-Live 所强调的连续智能体。

⁵Remote Procedure Call，远程过程调用。

2.2 状态、因果性与回声

因果系统只能使用 $u_{\leq t}$ ，但可通过有限 look-ahead L 换取更稳健的语音表征。算法延迟下界近似为

$$T_{\text{algo}} = T_{\text{chunk}} + T_{\text{lookahead}} + T_{\text{codec}} + T_{\text{decode}}. \quad (4)$$

减小 Δ 会提高决策频率，却增加 token rate、kernel launch 与调度压力。以 80–160 ms 为时间格的公开系统体现了典型折中：足够快以响应打断，又能在一个格内积累可辨别的声学证据 [7, 14]。

模型还必须区分“用户正在说”与“扬声器声音被麦克风重新采集”。传统 AEC⁶ 在波形域消除已知播放参考；模型侧还可显式接收 assistant history、自输出 token 或 echo flag。若只用单通道混合音频训练，模型可能把自己的声音误当用户指令。SALMONN-omni 将 echo cancellation 列为全双工核心场景 [15]，说明评估必须覆盖残余回声、双讲与不同声学路径，而不能只在干净的分离声道上测试。

2.3 双时间尺度控制

自然对话的控制延迟与任务推理延迟并不相同。快速环 π_f 每 Δ 毫秒运行，负责保持接触、决定是否让出话权；慢速环 π_s 可能运行数秒，负责搜索、规划或调用工具：

$$\pi_f : (u_t, m_{t-1}) \mapsto (a_t, z_t), \quad \pi_s : (q, M) \mapsto r, \text{quadr} \xrightarrow{\text{event}} m_{t+k}. \quad (5)$$

异步委派的难点在于如何将结果重新注入当前对话：结果可能已经过时，任务可能被用户取消，也可能与交互层先前的表述发生冲突。因此，事件至少应包含 task_id、上下文版本、创建时间、取消状态和来源信息；交互层收到结果后还应执行相关性判断，而不是无条件朗读。

3 公开模型路线：组件与设计轴

近年的公开模型虽然实现路径不同，却逐渐收敛到“共享时间轴、用户与助手双流、持续状态更新”这一基本结构。真正影响系统行为的是三组权衡：codec token 的生成统一性与高 token rate，channel fusion 的语义耦合能力与 overlap 干扰，以及 silence/control/action token 的控制精度与主干负担；这些设计轴可以解释 Moshi、SyncLLM、LSLM、Neural-FSM 和 DuplexSLA 的差异，也构成理解 GPT-Live 的候选机制空间。

3.1 音频表示：连续特征、semantic token 与 codec token

端到端语音 LLM 通常先把 16/24 kHz 波形降采样为低频表示。SoundStream 与 EnCodec 使用卷积编码器、残差向量量化 (RVQ) 和生成式解码器，将波形压缩为多 codebook 离散 token [16, 17]。若帧率为 f 、codebook 数为 K 、每本词表大小为 V_k ，原始离散码率为

$$R = f \sum_{k=1}^K \log_2 V_k \quad \text{bit/s}. \quad (6)$$

高层 codebook 往往承载较粗的语义和韵律，后续 codebook 补充声学细节。AudioLM 进一步分离 semantic token 与 acoustic token，以较低频语义保证长期一致性，以 codec code 重建音质 [18]。

VITA-Audio 用 multiple cross-modal token prediction 在一次 forward pass 中生成多个 audio token，作者报告 7B 规模下获得 3–5 倍 inference speedup [19]，说明 parallel acoustic decoding 是降低 time-to-first-audio 的重要方向。

Codec-token 路线可让 next-token prediction 同时覆盖理解与生成，但 token rate 很高。假设每秒 12.5 frame、8 个 codebook，即每秒 100 个 audio token；若双方流同时存在，backbone throughput 和 KV cache⁷ 增长会显著高于文本聊天。典型优化包括 delay pattern、hierarchical depth decoder、multi-token prediction 和 semantic/acoustic 分层。Moshi 用 Temporal Transformer 建模时间、Depth Transformer 在同一 frame 内展开多 codebook，并用时间对齐文本 Inner Monologue 改善语言质量 [7]。

另一条路线不把 codec id 注入 LLM 词表。LSLM 以 streaming SSL⁸ encoder 提供连续听觉特征、token-based TTS 生成语音；SALMONN-omni 则强调 codec-free token space 与 dynamic thinking [9, 15]。这种做法可降低主干需要学习的声学熵，却要求外部 speech decoder 在低延迟和高保真之间取得平衡。两类路线本质上是“主干承担多少声学细节”的分工差异。

近期综述将相关方法概括为“工程式同步”和“学习式同步”，也印证了模型端与系统端需要并行演进 [20]。

⁶Acoustic Echo Cancellation，声学回声消除。

⁷Key-Value cache，Transformer 在增量解码时缓存历史 token 的 Key 和 Value 表示，以避免重复计算。

⁸Self-Supervised Learning，自监督学习；此处指用无标注语音学习连续声学表示。

3.2 双方流如何进入主干

序列展平。SyncLLM 把真实时间切成 chunk，以同步标记把用户和助手事件序列化；OmniFlatten 用统一 flatten operation 覆盖模态对齐、半双工与全双工训练 [8, 21]。优点是复用标准 decoder-only LLM 和 next-token loss，缺点是序列次序可能人为打破“同一时刻”的对称性，且 token rate 膨胀。

平行流与多码本。Moshi 同时建模 user 与 assistant audio stream，dGSLM 更早使用双塔 Transformer 与 cross-attention，在两个声道并行生成语音、笑声和轮次动态 [7, 22]。平行表示最贴近物理过程，但 batch、cache 与 loss mask 都需认识二维的时间 $\times$ 通道结构。

Fusion 与 cross-attention。LSLM 比较了 early、middle 和 late fusion。2026 年的一项控制实验进一步表明，channel fusion 能够建立更强的语义联系，但在输入 overlap 时也更容易干扰正在生成的 context；把 user stream 作为 external memory、通过 cross-attention 读取则更加稳健，但问答能力相对偏弱 [9, 23]。因此，架构评估不能只看静态 spoken QA，还必须考察被打断后的 semantic recovery。

3.3 交互动作如何表达

最简单的做法是把静默也视为一种输出：如果模型在某个时间格生成 silence token，就表示它选择“继续听”。SyncLLM 和 Moshi 可以从用户、助手两条流的联合分布中隐式学习重叠与停顿。Neural-FSM 则让 LLM 显式输出 control token，驱动一个双状态 FSM⁹，决定开始回答、继续等待或接受打断；论文报告称，在仿真评测中，其平均响应延迟较半双工系统降低超过三倍，超过一半的交互能在 500 ms 内响应 [10]。显式控制更便于监督和审计，但过于粗粒度的状态机也可能限制交互的自然程度。

DuplexSLA 在共享的 160 ms 时间线上，联合解码助手音频与限速的文本动作通道，使规划和工具调用可以与语音输出并行进行 [14]。这代表一种更一般的语音-语言-动作（Speech-Language-Action, SLA）建模方式：音频负责低带宽的社交反馈，动作通道负责高精度的结构化控制。GPT-Live 官方只确认系统能够持续决定何时调用工具或发起异步委派；内部是否存在原生动作头，目前尚未披露。

表 1: 全双工公开路线的核心设计轴。表中“GPT-Live”仅列官方公开属性，问号表示未披露。

系统	输入组织	语音输出	交互控制	深层推理	主要特征
Moshi	双平行音频流	RVQ codec	隐式时序	单主干	Inner Monologue, 约 200 ms 实测
SyncLLM	时间 chunk 展平	HuBERT/语音单元	sync token	单主干	真实时钟同步
Neural-FSM	序列化感知事件	模块化 motor	control token	LLM 内	两状态 FSM
LSLM	streaming SSL	token TTS	融合后预测	单主干	early/middle/late fusion
OmniFlatten	flatten 序列	speech token	数据中学习	单主干	三阶段 post-training
SALMONN-omni	连续语音特征	独立生成模块	dynamic thinking	单主干	codec-free token space
DuplexOmni	流式音频/视频	流式语音	interaction layer	异步插件	交互-思考解耦
DuplexSLA	双流三通道	离散音频	action channel	同主干动作	共享 160 ms 时钟
GPT-Live	持续音频	持续语音	连续决策	异步委派	tokenizer/fusion 未公开

3.4 与端到端 Omni Model 的本质区别

两者并非互斥概念。Omni 强调模态统一：同一个模型能够理解并生成文本、图像或音频；full-duplex 强调时间与控制统一：新输入可以在生成过程中持续到达，模型能够感知 wall-clock，并在语音重叠时维持正确的因果状态。一个模型可以是 Omni 但仍按轮次交互，也可以具备 full-duplex 能力但只处理音频。

GPT-Live 进一步引入能力解耦：低延迟交互核心不必承担所有复杂推理。相比让超大 omni model 每 80–160 ms 都完成一次昂贵推理，该方案把“不能错过的帧级决策”和“可以等但需要聪明的任务”分配给不同计算路径。这既是产品体验设计，也是 serving 可行性的核心。

4 训练数据与后训练

4.1 为什么普通语音问答数据不够

Turn-based SFT¹⁰ 样本通常只有“完整用户音频 $\rightarrow$ 完整助手回答”，几乎没有重叠、犹豫、短反馈、抢话、取消和自我修正。模型即使 ASR、SQA¹¹、TTS 都很强，也只会等待明确句末。全双工训练样本应是双声道、共享时间轴事件序列

$$D = \{(u_{1:T}, a_{1:T}, c_{1:T}, s_{1:T}, q, r)\}, \quad (7)$$

⁹Finite-State Machine，有限状态机。

¹⁰Supervised Fine-Tuning，监督微调。

¹¹Spoken Question Answering，口语问答。

其中 c_t 是交互动作, s_t 是说话人/重叠状态, q, r 是可选的委派请求与返回。数据不仅要回答“说什么”, 还要监督“何时说、何时保持沉默、何时停止”。

真实双声道数据稀缺且隐私敏感。SyncLLM 以 212k 小时合成语音对话和 2k 小时真实对话训练时钟同步能力 [8]; DuplexChat-Pipe 从公共播客执行语言过滤、下载清洗、diarization、双人片段抽取、speech separation 与 restoration, 作者报告产出 282,634 小时英语和 132,723 小时日语数据 [24]。这些规模说明数据工程已成为模型能力的主要约束, 但自动分离会引入串音、错说话人、伪重叠与恢复伪影, 必须保留质量 tag, 而不能只报告小时数。

4.2 建议的渐进训练配方

公开工作普遍采用渐进式训练。OmniFlatten 依次进行模态对齐、半双工对话学习和全双工对话学习 [21]; Freeze-Omni 则冻结文本主干, 先接入语音输入输出模态, 再进行双工多任务训练 [25]。一个可复现的六阶段训练方案如下:

1. **Representation pretraining**: 优化重建质量、语义保真、流式因果性与低帧率, 建立 clean/noisy/echo 条件下的稳定表示。
2. **Speech-text alignment**: 混合 ASR、TTS、speech continuation、spoken QA, 让语音能力对齐文本主干; 对文本能力做 replay, 避免 catastrophic forgetting。
3. **Half-duplex dialogue**: 学习指令遵循、角色、语音风格和长上下文, 确保“说什么”先收敛。
4. **Synthetic duplex conversion**: 将文本对话扩展为带时间戳的双声道音频, 随机采样停顿、backchannel、重叠、barge-in 与网络延迟。
5. **Real duplex adaptation**: 用真实双声道对话校准时间分布、韵律和文化差异, 降低合成数据的模板偏置。
6. **Preference/RL**¹²: 针对打断精度、错误抢话、任务完成、说话冗余与恢复能力优化; 引入 safety interrupt 和 delegate policy。

损失可写为

$$\mathcal{L} = \lambda_a \mathcal{L}_{audio} + \lambda_t \mathcal{L}_{text} + \lambda_c \mathcal{L}_{control} + \lambda_{asr} \mathcal{L}_{align} + \lambda_s \mathcal{L}_{safety} + \lambda_d \mathcal{L}_{delegate}. \quad (8)$$

不同 loss 对应的 token 数量级差异很大。audio token 的密度远高于 control token; 若简单按 token 平均, 模型会优先优化声学细节而忽略稀有但关键的正确打断。因此应对 control/action event 进行重加权, 并按场景报告 precision/recall, 而非只看总 loss。

4.3 数据流水线的可量化质量控制

建议为每个音频片段保留以下质量字段: 说话人纯度、重叠比例、信噪比、残余回声、说话人分离置信度、语音分离伪影分数、语言与口音、轮次切换间隔、简短应答标签、隐私与安全标签以及许可来源。关键门槛包括: (1) 双声道互相关过高时判定为串音泄漏; (2) ASR 转写与说话人时间轴不一致时降低样本权重; (3) 将极端负间隔和长时间重叠单独分桶; (4) 在训练集与评测集之间, 对同一播客、同一说话人和近重复音频进行去重。

对于合成数据, 需要进行双向校准: 先由真实数据给出轮次切换间隔、重叠时长和简短应答频率的目标分布, 合成器再按照这些分布采样; 训练完成后, 还要检查模型输出是否偏离真实分布。单纯追求更低的响应延迟会诱导模型频繁抢话, 因此必须同时把错误打断率作为保护指标。

5 Stateful Serving: 把 audio frame 按时送达

实时语音 serving 的容量上限不是单位时间完成多少 request, 而是多少并发 session 能持续在各自 playout deadline 前生成 audio frame; 平均 RTF¹³ 小于 1 仍可能掩盖 burst prefill 造成的连续 deadline miss。长会话还会使 KV cache 随 audio token 持续增长, 因此 context compaction 必须移出 live path, 并通过 replacement instance 的后台 prefill、并行运行和安全 cut-over 完成无感 handoff。

5.1 容量单位从 request 转变为 session-frame

文本 serving 可以用单请求 latency 换 throughput; 实时语音却不能让 session 等待长 prefill。

[F1]OpenAI 明确以“多少并发 session 能持续按时处理 frame”衡量容量, 而非 GPU request throughput[1]。

¹²Reinforcement Learning, 强化学习。

¹³Real-Time Factor, 实时因子; 通常指处理一段音频所需时间与该段音频时长之比。

设活跃会话集合为 S ，第 i 个会话 frame 预算为 Δ_i ，则更合理的 SLO¹⁴ 是

$$\text{DMR} = \frac{\sum_{i,t} \mathbf{1}[Q_{i,t} + C_{i,t} > \Delta_i]}{\sum_i T_i}, \quad (9)$$

$$\text{GoodSessions} = \sum_i \mathbf{1}[\text{DMR}_i < \epsilon \wedge P_{99}(J_i) < J_{max}], \quad (10)$$

这里分别衡量 deadline-miss ratio (DMR)¹⁵ 和满足 jitter 门槛的并发会话数。即使平均 real-time factor (RTF) 低于 1，也不能保证交互体验，因为突发的 context prefill 仍可能导致连续 frame miss。

调度器应采用 deadline-aware continuous batching：按照最近 playout deadline 对 audio frame 排序；限制每批 context prefill 的规模；为 audio decode 预留算力；将后台 context compaction、转写修订和委派模型 prefill 放入低优先级队列。负载过高时，应依次减少非关键遥测、降低 acoustic refinement 强度、延后后台任务，最后才考虑牺牲交互核心的质量。

5.2 KV cache 与连续上下文

Transformer-XL 的 segment recurrence 是较早的跨片段记忆方案 [26]，但实时语音还要求状态与播放时间严格一致。

Transformer 的每层 KV cache 随时间增长。粗略显存为

$$M_{KV} \approx 2LNH_{kv}d_hb, \quad (11)$$

其中 L 是层数、 N 是累计 token、 H_{kv} 是 KV head 数、 d_h 是 head dimension、 b 是每元素字节。音频高 token rate 与长会话叠加，使 cache 成为 session 容量瓶颈。PagedAttention 通过分页降低动态 cache 的碎片和复制，FlashAttention 通过 IO-aware tiling 降低注意力的 HBM 访问 [27, 28]；但它们不自动解决无限上下文。

可采用分层记忆：(1) 最近 W 秒保留原始双流 token，保证打断处理和指代消解；(2) 将较早的语音压缩为最终转写和事件摘要；(3) 为工具结果保留结构化 provenance；(4) 把用户偏好写入独立的长期状态。压缩并非无损操作，它会改变 token prefix 并使旧 KV cache 失效。[F1]OpenAI 将 context compaction 移出 live path，在后台完成 replacement instance 的 prefill 后再切换 [1]，从而避免主实例因长上下文 prefill 而停顿。

5.3 无缝实例切换

有状态 session 无法像普通 HTTP 请求一样任意负载均衡。官方描述的切换过程是：启动并预热 replacement；用压缩后上下文 prefill；新旧实例并行运行；在安全边界 cut over。这个过程同时解决上下文压缩、实例维护和故障恢复。

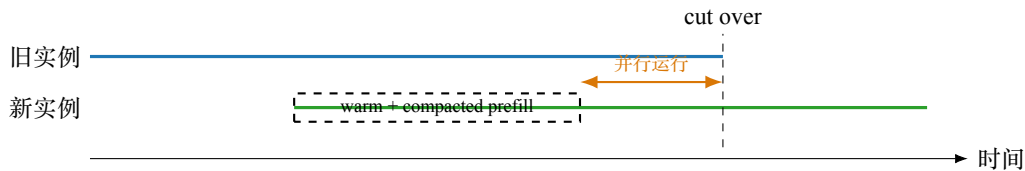


图 3: 有状态推理实例切换。切换点应位于可恢复的声学边界，并对重复/缺失 frame 做去重。

工程上还要处理三类竞态条件：工具结果恰好在实例切换期间到达；最终转写对旧文本进行修订；用户在切换边界插话。建议为所有事件附加单调递增的 sequence number 和 context epoch；新实例只接受 epoch 匹配的状态变更，并按照 playout timestamp 对 media frame 去重。

5.4 异步委派的 serving 设计

[F1]为降低复杂任务的首次响应延迟，系统会预先创建并填充 frontier-model session，保持稳定的 session affinity，并利用 prompt caching[1]。委派耗时可分解为

$$T_{delegate} = T_{route} + T_{queue} + T_{prefill_delta} + T_{reason} + T_{tool} + T_{return}. \quad (12)$$

预填充只能减少 $T_{prefill_delta}$ ；推理强度、工具模式定义的大小和串行工具调用轮数仍然决定长尾延迟。因此，交互层需要采用渐进式反馈：先用简短应答告知用户任务正在处理，结果到达后再选择合适时机插入；如果用户已经更换话题，则取消后台任务或降低其优先级。

¹⁴Service-Level Objective, 服务等级目标。

¹⁵Deadline-Miss Ratio, 未能在 deadline 前完成的 frame 占比。

Speculative decoding 可以由小模型先生成 draft，再交给大模型并行验证，在不改变目标分布的前提下减少串行 decoding step[29]；但它更适合 cognition path，不能替代 interaction path 中具有 hard deadline 约束的调度。

安全策略也必须跨层一致。GPT-Live 系统卡称安全系统随对话持续检查输入输出，可 steer、interrupt、播放安全响应或结束会话 [12]。这意味着安全控制不能只位于委派模型末端，否则在长推理返回前，实时层可能已经生成不当语音。

6 实时网络：WebRTC、WARP 与 playout clock

6.1 为什么不是“换成 WebSocket 就够了”

WebRTC 将媒体、NAT¹⁶ 穿透、拥塞控制、安全和 data channel 组合成浏览器实时通信栈。标准 WebRTC 中，媒体通常经 SRTP¹⁷，非媒体 data channel 使用 SCTP over DTLS over ICE/UDP¹⁸[30, 31]。音频帧与工具事件的可靠性需求不同：晚到的旧音频包通常不值得重传，而工具结果、状态变更需要可靠和有序。将两者都放进一个可靠字节流会产生 head-of-line blocking。

端到端感知延迟可以写为

$$T_{e2e} = T_{capture} + T_{uplink} + T_{jitter,in} + T_{model} + T_{downlink} + T_{jitter,out} + T_{playout}. \quad (13)$$

如果模型侧节省的 20 ms 被 300 ms 的 connection setup 或 jitter buffer 抵消，用户就感知不到这部分优化。反过来，jitter buffer 过小又会在弱网环境下增加缺帧。系统应根据近期 packet loss、RTT¹⁹ 和 clock drift 动态调整 playout 策略，而不是为全球网络设置同一个固定参数。

SimulTron 的端侧同步语音到语音翻译同样联合设计了因果编码器、可调固定延迟和流式声码器 [32]，说明评估延迟时必须同时计入模型算法和音频播放路径。

6.2 从 6 RTT 到 1 RTT 的建连优化

[F1]OpenAI 将相关 WebRTC 优化统称为 WARP，称媒体与 data channel 建立从 6 个 round trip 降为 1 个，组合 SPED²⁰、DTLS 1.3、SNAP²¹ 和预协商 data channel[1]。DTLS 1.3 本身使用更短的握手模式 [33]；SPED 将 DTLS handshake data/ack 嵌入 STUN Binding Request/Response；SNAP 将 SCTP 初始化参数嵌入 SDP²² offer/answer，从而减少 data channel 的串行往返 [34, 35]。后两者截至本文写作时仍是 Internet-Draft，不能当作稳定 RFC²³。

Instant Connect 更进一步预协商 SDP，使客户端可用单个 UDP packet 启动会话 [1]。这要求服务端提前分配或恢复足够的连接状态，也引入防重放、资源滥用和预留过期问题。低 RTT 不是免费午餐：状态预热越激进，空连和攻击流量造成的成本越高。

6.3 丢包、漂移与音频追赶

client sampling clock、server generation clock 和 speaker playout clock 并不完全一致。若生成略慢，buffer 逐渐耗尽；若生成略快，buffer 累积导致交互越来越迟。官方文章提到 audio time stretching/catch-up[1]：在小范围内调整 playout rate 或选择性丢弃低价值片段，使延迟回到目标区间。实现时应限制 stretching ratio，并在 voice activity boundary 操作，避免音高和韵律畸变。

弱网测试至少覆盖独立随机丢包与 burst loss、上/下行不对称、RTT 突增、NAT rebinding、Wi-Fi/蜂窝切换、客户端后台恢复及音频设备切换。评价不仅是 packet loss 后的 ASR WER²⁴，还包括 assistant 是否重复、抢话、忘记用户中途修正，以及工具事件是否与音频状态一致。

¹⁶Network Address Translation，网络地址转换；NAT traversal 指跨越地址转换设备建立端到端连接。

¹⁷Secure Real-time Transport Protocol，安全实时传输协议。

¹⁸SCTP：Stream Control Transmission Protocol，流控制传输协议；DTLS：Datagram Transport Layer Security，数据报传输层安全协议；ICE：Interactive Connectivity Establishment，交互式连接建立；UDP：User Datagram Protocol，用户数据报协议。

¹⁹Round-Trip Time，往返时延，即数据从发送端到接收端再返回所需的时间。

²⁰STUN Protocol for Embedding DTLS；其中 STUN 为 Session Traversal Utilities for NAT。该草案把 DTLS handshake data 与 acknowledgement 嵌入 STUN Binding 消息。

²¹SCTP Negotiation Acceleration Protocol，把 SCTP 初始化参数放入 SDP offer/answer，以减少 data channel 建立的串行往返。

²²Session Description Protocol，会话描述协议。

²³Request for Comments，互联网工程与协议领域的标准、规范或信息性文档系列。

²⁴Word Error Rate，词错误率，通常按替换、删除和插入错误相对参考词数计算。

7 评测体系：从模型分数到连续交互体验

7.1 四层评测框架

Full-Duplex-Bench 将核心交互行为拆为停顿处理、简短应答、轮次切换和打断管理 [36]；v2 进一步加入多轮阶段性目标、快慢说话节奏、纠错、实体跟踪与安全任务 [37]。结合线上系统需求，建议从以下四个层面进行评测：

1. 声学/语义：ASR WER、spoken QA accuracy、speaker similarity、MOS²⁵/UTMOS、情绪与韵律遵循、噪声/口音鲁棒性。
2. 交互行为：起始响应延迟、停顿/重叠分布、简短应答的准确率与召回率、用户插话后的停声延迟、错误打断率，以及恢复后的语义一致性。
3. 智能体任务：工具调用正确率、参数精确匹配率、委派触发的准确率与召回率、任务完成率、取消指令遵从率、结果时效性和来源信息正确率。
4. 系统服务质量：connection RTT、首段音频延迟、audio frame deadline-miss ratio、jitter、playout buffer underrun、handoff gap、单 session GPU 显存、每块 GPU 可稳定承载的 session 数，以及每分钟成本。

人类 turn-taking 本身存在文化差异 [38]，因此不能以单一“越快越好”作为自然性目标。应报告 transition gap 的完整分布，并让人评同时评价响应相关性、是否唐突、是否啰嗦和对打断的尊重程度。

7.2 关键指标定义

设用户在 t_b 时刻开始插话，助手在 t_s 时刻停止可听语音，则停声延迟为 $t_s - t_b$ ；只有当用户输入确实构成有效的语义打断时，才记为真正例。由此定义

$$P_{int} = \frac{N_{correct\ stop}}{N_{all\ stop}}, \quad R_{int} = \frac{N_{correct\ stop}}{N_{valid\ barge-in}}. \quad (14)$$

只优化 R_{int} ，可能导致咳嗽或简短应答也让模型停止发言；只优化 P_{int} ，模型又可能显得一直“抢着说”。因此，应同时报告 F_1 、停声延迟分位数和恢复成功率。

委派应以 counterfactual 方式评价：对同一任务比较不委派、同步委派、异步委派。指标包括 interaction stall、最终正确率、首个有用反馈、工具往返数和用户取消后的浪费算力。异步方案的价值不是让最终答案必然更快，而是在慢任务期间保持对话可用。

7.3 实验矩阵与归因原则

为避免混淆因素，建议固定 backbone、训练 token、双工/真实数据比例、codec、硬件与网络 trace，只改变一个设计轴。表中的 FIFO²⁶ 与 EDF²⁷ 是两类调度基线，核心消融如下：

表 2：建议的可归因消融实验。

设计轴	对照组	主要指标与预期风险
用户流路由	通道融合 / cross-attention / 门控混合	SQA、重叠后语义干扰、停声延迟
时间粒度	40/80/160/320 ms	DMR、打断延迟、token throughput、语音自然度
动作表达	仅 silence token / FSM control token / action channel	控制精度、工具参数精确匹配、主干负担
训练方案	直接全双工 / 三阶段 / 六阶段	收敛、文本能力遗忘、真实交互评分
交互-思考	单模型 / 同步委派 / 异步委派	交互停顿、任务正确率、成本、取消浪费
上下文管理	sliding window / 摘要 / 后台 handoff	长会话一致性、handoff gap、KV cache 显存
调度策略	FIFO / continuous batching / EDF+ 算力预留	每 GPU 稳定 session 数、DMR、P99 jitter
网络	标准 WebRTC / 1-RTT 优化 / 预连接	connect latency、安全与空连成本

²⁵Mean Opinion Score，平均主观意见分，通常由听者评价语音自然度或质量；UTMOS 是自动预测 MOS 的神经网络评估器。

²⁶First In, First Out，先进先出。

²⁷Earliest Deadline First，最早截止期优先。

每项结果至少报告均值、P50/P95/P99²⁸、置信区间与分场景样本量。语音模型的随机性和网络 trace 都会放大方差；若只报告一次 demo 或平均 latency，无法支持架构结论。

8 证据约束下的 GPT-Live 架构重建

公开证据可以确定 GPT-Live 在发声时持续监听、按细粒度时钟作出交互决策，并把复杂任务异步委派给 frontier model；但这些事实不足以推出其具体 tokenizer、fusion 位置或 action head。满足这些约束的最小重建是一种混合架构：media fast path 承载连续音频，stateful interaction core 维护双流因果状态，action channel 触发工具和委派，异步 cognition plane 负责慢速推理，而 authoritative state 负责转写、安全与 context compaction。

8.1 哪些部分可以确定

综合官方材料，可以确定以下系统性质：**[F1]** (1) 语音模型在发声时继续监听，不以传统轮次检测器作为音频处理的推进条件；(2) 模型每秒多次决定何时说、听、暂停、接受打断、调用工具或发起委派；(3) 复杂任务可以异步委派，同时保持交互不中断；(4) 媒体快速路径与应用逻辑由异步 RPC 隔离；(5) 会话推理具有持久状态，并支持后台预填充和实例切换；(6) 上下文压缩不会阻塞实时路径；(7) WebRTC/WARP、丢包处理与时钟漂移校正共同决定端到端体验；(8) 系统容量以会话音频帧能否按时完成来衡量 [1, 2]。

这些性质共同排除了两种过度简化的解释。第一，GPT-Live 不太可能只是“更快的 ASR、LLM 与 TTS”，因为这类管线依赖离散轮次，无法自然处理系统发声期间到达的新输入。第二，它也不应被理解为“由一个超大模型承担全部任务”，因为官方明确说明系统会把复杂任务异步交给前沿模型处理。

8.2 最可信但尚未证实的内部机制

[H]交互 core 很可能具有等价于双流时间格的内部状态：每个 chunk 编码新用户音频和近期 self-output，更新 causal state，再生成 assistant acoustic unit 与 interaction action。该表示可能像 Moshi 的 parallel stream、SyncLLM 的同步序列、LSLM 的 continuous encoder 加 fusion，也可能是未公开的新结构；公开信息不足以二选一。

[H]为了在自身说话时理解用户，系统必须获得至少一种 self-speech reference：波形级 AEC 后音频、模型生成 token、播放时间戳或其组合。否则 residual echo 会和真实用户输入混淆。官方提到音频时钟与播放追赶，但未披露 echo reference 如何进入模型。

[H]“交互模型委派 frontier model”可以通过结构化事件实现：交互模型输出 delegate intent 和压缩后的 query，应用服务器把最终确认的 context 及 tool schema 送入已经预热的 session；返回结果写入 event stream，再由交互模型选择合适时机将其转化为自然口语。这个过程既不要求语音模型暴露内部思维，也不要求两个模型共享 KV cache。

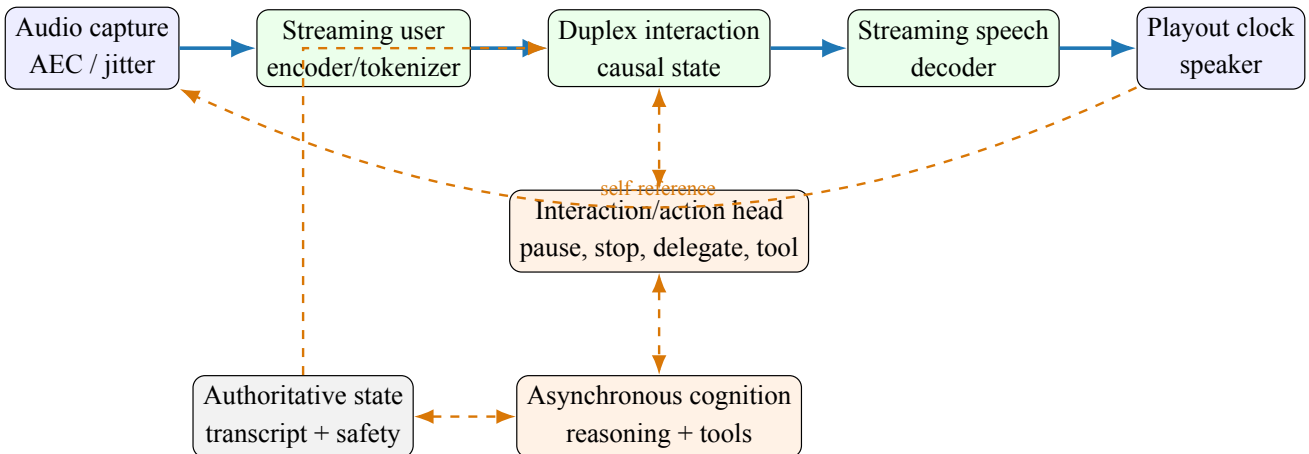


图 4: 可实现 GPT-Live 公开行为的最小架构重建。绿色语音内部结构与橙色动作头均属假设；官方确认的是持续交互、异步委派和系统边界。

8.3 “混合架构”比“纯端到端”更准确

在感知-发声路径上，端到端学习可以保留韵律、重叠和非语言信号；在复杂任务路径上，模块化委派允许使用更强推理模型、工具权限和独立扩缩容；在 transport/state 层，确定性协议和显式状态机仍不可替代。因此

²⁸ 分别表示第 50、95 和 99 百分位数；P99 常用于观察系统长尾延迟。

GPT-Live 更像“端到端交互模型嵌入模块化实时系统”，而不是对所有组件做单模型吞并。

这种拆分还能带来安全和成本方面的优势：小而快的交互模型可以高频运行，昂贵模型只在必要时启动；工具权限集中在可审计的认知平面；安全控制可以同时作用于实时输出和后台结果。代价则是跨层一致性、过期结果处理、委派策略和会话状态恢复会更加复杂。

9 一个可复现的开源实现方案

9.1 最小可行架构

建议以三个独立服务实现研究原型：

1. **Realtime gateway**: WebRTC ingress/egress、Opus、AEC/jitter buffer、播放时间戳、连接与弱网遥测。媒体包进入有界 ring buffer，应用事件进入可靠 data channel。
2. **Duplex inference worker**: 包括 causal audio encoder、文本 LLM backbone、user-stream adapter、assistant audio decoder 和 action head。worker 保持固定的 session affinity，每 80 或 160 ms 执行一次 inference tick。
3. **Agent 与 state service**: 负责最终转写、conversation event log、tool runtime、reasoning model、context compaction、安全策略和 instance handoff 协调。

原型可以从 Moshi 开源栈的语音主干出发 [7]，增加独立的 action vocabulary 和 asynchronous tool bus；也可以采用 Freeze-Omni/LSLM 的路线，冻结文本 LLM，并接入 streaming encoder 和轻量 audio decoder，以降低训练成本 [9, 25]。第一阶段不宜同时追求视频、超长 context 和多工具能力，应先验证真实 double-talk 条件下的 audio frame deadline 和打断行为。

9.2 帧级调度伪代码

Algorithm 1 Deadline-aware duplex session loop

```

1: for all active session  $s$  in earliest-deadline order do
2:    $u_t \leftarrow \text{readAudioChunk}(s, \Delta)$ 
3:    $e_t \leftarrow \text{drainReadyEvents}(s, \text{contextEpoch})$ 
4:    $(a_t, z_t, m_t) \leftarrow \text{duplexStep}(u_t, e_t, m_{t-1})$ 
5:   if  $z_t$  requests delegation and no equivalent task is active then
6:      $\text{enqueueAsync}(\text{taskId}, \text{contextEpoch}, \text{query})$ 
7:   end if
8:   if safety permits  $a_t$  and playout deadline remains then
9:      $\text{sendAudio}(s, a_t, \text{timestamp})$ 
10:  else
11:     $\text{applyGracefulDegradation}(s)$ 
12:  end if
13: end for
14: run background prefill, compaction, and telemetry with residual budget

```

调度循环需要避免三个隐蔽的阻塞点：tokenizer 或 audio codec 在 CPU 上串行执行；等待最终 ASR transcript 后才更新状态；在 media coroutine 中错误地同步等待 tool task。所有耗时操作都应通过 event 返回结果，media loop 只消费已经就绪的 event。

9.3 分阶段实验计划

阶段 A：离线可学习性。固定 7B–9B 文本 backbone 和 audio representation，在训练 token 数相同的条件下比较 channel fusion、cross-attention 和 gated fusion。训练集同时包含 half-duplex 对话、合成 overlap speech 和小规模真实双声道数据。验收标准包括：spoken QA 相对 half-duplex baseline 的退化不超过预设阈值；有效 barge-in 的 recall 提升且 false interruption rate 不恶化；存在 echo 时不发生 self-trigger。

阶段 B：单机实时性。在固定 GPU 上逐步增加并发 session，记录每个 tick 的 Q/C 、DMR、P95/P99 jitter、KV cache 显存和 audio buffer underrun。比较 FIFO、continuous batching、earliest deadline first (EDF) 以及预留 20% 算力等策略。验收不以最高 token throughput 为准，而以每块 GPU 能稳定承载的 session 数为准。

阶段 C：异步认知。接入一个文本推理模型和三类工具：低延迟查询、长延迟搜索、多轮参数补全。对比不委派、阻塞式委派和异步委派三种模式，检查工具结果正确率、交互停顿、结果过期率、任务取消后的算力浪费，以及用户的主观等待感受。

阶段 D：长会话与故障。运行 30、60 和 120 分钟的长 session，主动注入 instance restart、network handoff、context compaction、transcript revision 和 tool-result race。验收标准包括：instance handoff 时不出现语音重复或缺失；context epoch 保持一致；用户关键事实得到保留；GPU 显存不随 session 时长无限增长。

9.4 资源与延迟预算示例

以 160 ms 为一个 inference tick 时，可以采用如下参考预算：uplink 与 jitter buffer 40 ms，streaming encoder 15 ms，interaction backbone 的 queueing 与 compute 45 ms，audio head 与 codec 20 ms，downlink、jitter buffer 和 playout 40 ms。这不是 GPT-Live 的公开参数，只是一个工程起点。预算必须使用 P95/P99 而不是平均值进行验证，并在跨地域、低端客户端和 5% burst loss 条件下重新测试。

成本模型应按会话分钟分解为 interaction GPU、cognition GPU、音频带宽、tool 调用和 idle reservation。若委派率 p_d 、平均 reasoning 时间 T_r 、交互 core 每分钟成本 C_f ，则粗略成本为

$$C_{min} = C_f + p_d T_r C_s + C_{net} + C_{tool} + C_{idle}. \quad (15)$$

降低 p_d 可以节省成本，但也可能损害任务正确率；因此，应结合委派准确率、召回率和最终任务成功率共同调优。

10 局限性、开放问题与结论

10.1 仍然未知的关键细节

本文不能回答 GPT-Live 的具体参数量、训练数据、codec、audio token rate、fusion 层位置、是否联合输出 action token、回声参考注入方式、推理并行策略和部署规模。这些信息从未被 OpenAI 官方公开。报告中的图 4 是满足公开行为的一种最小实现，不是泄露或反向工程结论。

现有论文之间也很难公平横比。Moshi 的“约 200 ms”、Neural-FSM 的“500 ms 内响应”、GPT-4o 的“平均 320 ms”使用不同任务、网络、硬件和延迟起点。本文保留这些数字用于说明各自设计目标，不把它们排成统一排行榜。

10.2 研究前沿

未来最值得关注的方向包括：(1) 在强语义整合和抗 overlap 干扰之间，自适应地选择 user-stream routing；(2) 将 speech、language 和 action 放在统一 clock 上，同时避免高频 acoustic token 淹没稀疏 action；(3) 利用真实双声道数据训练能够适应不同文化 turn-taking 习惯的模型；(4) 为长 session compaction 建立事实保真和可逆审计机制；(5) 联合优化 inference tick、GPU scheduling、network jitter 和 playout clock；(6) 建立覆盖 stale result、task cancellation 和 provenance tracking 的 asynchronous delegation benchmark。

10.3 结论

GPT-Live 的核心并不是某个神秘的声学模块，而是将语音智能体重构为持续运行的实时系统。与轮次式全模态模型相比，它允许新输入在生成期间持续进入模型状态，并让交互策略与现实时间同步；与早期的单模型全双工方案相比，它进一步把复杂认知任务放到异步路径中处理，并通过有状态推理服务、后台上下文压缩与实例切换、低往返时延 WebRTC，将模型能力转化为稳定的产品体验。

因此，复现 GPT-Live 不应从“训练一个更大的语音 LLM”开始，而应同时建立四个闭环：表示闭环保证说和听能够并行，控制闭环学习停顿与打断，认知闭环安全地执行异步委派，系统闭环确保每个音频帧都能在播放时限之前到达。公开论文已经给出了这些组件的大部分候选机制；真正困难、也最有研究价值的，是让它们在同一在线系统中稳定协同。

参考文献

- [1] OpenAI. How we built a realtime system for responsive voice AI in six months, August 2026. URL <https://openai.com/index/continuous-voice-interaction-with-gpt-live/>. Accessed 2026-08-07.
- [2] OpenAI. Introducing GPT-Live, August 2026. URL <https://openai.com/index/introducing-gpt-live/>. Accessed 2026-08-07.

- [3] OpenAI. GPT-4o system card, August 2024. URL <https://openai.com/index/gpt-4o-system-card/>. Accessed 2026-08-07.
- [4] Jin Xu, Zhifang Guo, Jinzheng He, Hangrui Hu, Ting He, Shuai Bai, Keqin Chen, Jialin Wang, Yang Fan, Kai Dang, Bin Zhang, Xiong Wang, Yunfei Chu, and Junyang Lin. Qwen2.5-Omni technical report. *arXiv preprint arXiv:2503.20215*, 2025. URL <https://arxiv.org/abs/2503.20215>.
- [5] Aohan Zeng, Zhengxiao Du, Mingdao Liu, Kedong Wang, Shengmin Jiang, Lei Zhao, Yuxiao Dong, and Jie Tang. GLM-4-Voice: Towards intelligent and human-like end-to-end spoken chatbot. *arXiv preprint arXiv:2412.02612*, 2024. URL <https://arxiv.org/abs/2412.02612>.
- [6] Zhifei Xie and Changqiao Wu. Mini-omni: Language models can hear, talk while thinking in streaming. *arXiv preprint arXiv:2408.16725*, 2024. URL <https://arxiv.org/abs/2408.16725>.
- [7] Alexandre Défossez, Laurent Mazaré, Manu Orsini, Amélie Royer, Patrick Pérez, Hervé Jégou, Edouard Grave, and Neil Zeghidour. Moshi: A speech-text foundation model for real-time dialogue. *arXiv preprint arXiv:2410.00037*, 2024. URL <https://arxiv.org/abs/2410.00037>.
- [8] Bandhav Veluri, Benjamin N. Peloquin, Bokai Yu, Hongyu Gong, and Shyamnath Gollakota. Beyond turn-based interfaces: Synchronous LLMs as full-duplex dialogue agents. In *Proceedings of EMNLP*, pages 21390–21402, 2024. doi: 10.18653/v1/2024.emnlp-main.1192. URL <https://aclanthology.org/2024.emnlp-main.1192/>.
- [9] Ziyang Ma, Yakun Song, Chenpeng Du, Jian Cong, Zhuo Chen, Yuping Wang, Yuxuan Wang, and Xie Chen. Language model can listen while speaking. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025. URL <https://arxiv.org/abs/2408.02622>.
- [10] Peng Wang, Songshuo Lu, Yaohua Tang, Sijie Yan, Wei Xia, and Yuanjun Xiong. A full-duplex speech dialogue scheme based on large language model. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/180d4373aca26bd86bf45fc50d1a709f-Abstract-Conference.html.
- [11] Muye Huang, Lingling Zhang, Xingyu Yu, Lei Shi, Zhanyu Ma, Jun Xu, Jiuchong Gao, Jinghua Hao, Renqing He, and Jun Liu. Duplexomni: Real-time listening, seeing, thinking, and speaking for full-duplex interaction. *arXiv preprint arXiv:2606.09186*, 2026. URL <https://arxiv.org/abs/2606.09186>.
- [12] OpenAI. GPT-Live system card, August 2026. URL <https://deploymentsafety.openai.com/gpt-live>. Accessed 2026-08-07.
- [13] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL <https://arxiv.org/abs/1706.03762>.
- [14] Haoyang Zhang, Jun Chen, Donghang Wu, Yuxin Li, Yuxin Zhang, Xiangyu Tony Zhang, Che Liu, Qingjian Lin, Yizhou Peng, Hexin Liu, Eng Siong Chng, Chao Yan, Boyong Wu, Yechang Huang, Xuerui Yang, and Fei Tian. Duplexsla: A full-duplex spoken language model with synchronized speech, language, and action. *arXiv preprint arXiv:2605.20755*, 2026. URL <https://arxiv.org/abs/2605.20755>.
- [15] Wenyi Yu, Siyin Wang, Xiaoyu Yang, Xianzhao Chen, Xiaohai Tian, Jun Zhang, Guangzhi Sun, Lu Lu, Yuxuan Wang, and Chao Zhang. SALMONN-omni: A standalone speech LLM without codec injection for full-duplex conversation. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/233aee920dab065709145371b5900b8f-Abstract-Conference.html.
- [16] Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. Soundstream: An end-to-end neural audio codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 30:495–507, 2022. URL <https://arxiv.org/abs/2107.03312>.
- [17] Alexandre Défossez, Jade Copet, Gabriel Synnaeve, and Yossi Adi. High fidelity neural audio compression. *arXiv preprint arXiv:2210.13438*, 2022. URL <https://arxiv.org/abs/2210.13438>.
- [18] Zalán Borsos, Raphaël Marinier, Damien Vincent, Eugene Kharitonov, Olivier Pietquin, Matt Sharifi, Dominik Roblek, Olivier Teboul, David Grangier, Marco Tagliasacchi, and Neil Zeghidour. AudioLM: A language modeling approach to audio generation. *arXiv preprint arXiv:2209.03143*, 2022. URL <https://arxiv.org/abs/2209.03143>.
- [19] Zuwei Long, Yunhang Shen, Chaoyou Fu, Heting Gao, Lijiang Li, Peixian Chen, Mengdan Zhang, Hang Shao, Jian Li, Jinlong Peng, Haoyu Cao, Ke Li, Rongrong Ji, and Xing Sun. VITA-Audio: Fast interleaved cross-modal token generation for efficient large speech-language model. *arXiv preprint arXiv:2505.03739*, 2025. URL <https://arxiv.org/abs/2505.03739>.
- [20] Yuxuan Chen and Haoyuan Yu. From turn-taking to synchronous dialogue: A survey of full-duplex spoken language models. *arXiv preprint arXiv:2509.14515*, 2025. URL <https://arxiv.org/abs/2509.14515>.
- [21] Qinglin Zhang, Luyao Cheng, Chong Deng, Qian Chen, Wen Wang, Siqi Zheng, Jiaqing Liu, Hai Yu, Chaohong Tan, Zhihao Du, and Shiliang Zhang. Omniflatten: An end-to-end GPT model for seamless voice conversation. *arXiv preprint arXiv:2410.17799*, 2024. URL <https://arxiv.org/abs/2410.17799>.
- [22] Tu Anh Nguyen, Eugene Kharitonov, Jade Copet, Yossi Adi, Wei-Ning Hsu, Ali Elkahky, Paden Tomasello, Robin Algayres, Benoit Sagot, Abdelrahman Mohamed, and Emmanuel Dupoux. Generative spoken dialogue language modeling. *arXiv preprint arXiv:2203.16502*, 2022. URL <https://arxiv.org/abs/2203.16502>.
- [23] Hui Lu, Xueyuan Chen, Huimeng Wang, Shuhai Peng, Shiyin Kang, Xixin Wu, and Zhiyong Wu. How should LLMs listen while speaking? a study of user-stream routing in full-duplex spoken dialogue. *arXiv preprint arXiv:2605.10199*, 2026. URL <https://arxiv.org/abs/2605.10199>.
- [24] Wataru Nakata, Yuki Saito, and Hiroshi Saruwatari. Duplexchat: Constructing speaker-separated full-duplex dialogue speech at scale for spoken dialogue language modeling. *arXiv preprint arXiv:2607.04941*, 2026. URL <https://arxiv.org/abs/2607.04941>.

- [25] Xiong Wang, Yangze Li, Chaoyou Fu, Yunhang Shen, Lei Xie, Ke Li, Xing Sun, and Long Ma. Freeze-omni: A smart and low latency speech-to-speech dialogue model with frozen LLM. *arXiv preprint arXiv:2411.00774*, 2024. URL <https://arxiv.org/abs/2411.00774>.
- [26] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. *arXiv preprint arXiv:1901.02860*, 2019. URL <https://arxiv.org/abs/1901.02860>.
- [27] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM Symposium on Operating Systems Principles*, 2023. URL <https://arxiv.org/abs/2309.06180>.
- [28] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL <https://arxiv.org/abs/2205.14135>.
- [29] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of ICML*, 2023. URL <https://arxiv.org/abs/2211.17192>.
- [30] Harald Alvestrand. Overview: Real-time protocols for browser-based applications. Technical Report RFC 8825, Internet Engineering Task Force, 2021. URL <https://datatracker.ietf.org/doc/rfc8825/>.
- [31] Randell Jesup, Salvatore Loreto, and Michael Tüxen. WebRTC data channels. Technical Report RFC 8831, Internet Engineering Task Force, 2021. URL <https://datatracker.ietf.org/doc/rfc8831/>.
- [32] Alex Agranovich, Eliya Nachmani, Oleg Rybakov, Yifan Ding, Ye Jia, Nadav Bar, Heiga Zen, and Michelle Tadmor Ramanovich. Simultron: On-device simultaneous speech-to-speech translation. *arXiv preprint arXiv:2406.02133*, 2024. URL <https://arxiv.org/abs/2406.02133>.
- [33] Eric Rescorla, Hannes Tschofenig, and Nagendra Modadugu. The datagram transport layer security (DTLS) protocol version 1.3. Technical Report RFC 9147, Internet Engineering Task Force, 2022. URL <https://datatracker.ietf.org/doc/rfc9147/>.
- [34] Philipp Hancke, Eric Rescorla, and Jonny Rose. SPED: STUN protocol for embedding DTLS. Technical Report draft-hancke-webrtc-sped-00, Internet Engineering Task Force, 2026. URL <https://datatracker.ietf.org/doc/draft-hancke-webrtc-sped/>. Internet-Draft, work in progress.
- [35] Philipp Hancke and Eric Rescorla. SNAP: Accelerating WebRTC data channel establishment. Technical Report draft-hancke-tsvwg-snap-00, Internet Engineering Task Force, 2026. URL <https://datatracker.ietf.org/doc/draft-hancke-tsvwg-snap/>. Internet-Draft, work in progress.
- [36] Guan-Ting Lin, Jiachen Lian, Tingle Li, Qirui Wang, Gopala Anumanchipalli, Alexander H. Liu, and Hung-yi Lee. Full-duplex-bench: A benchmark to evaluate full-duplex spoken dialogue models on turn-taking capabilities. *arXiv preprint arXiv:2503.04721*, 2025. URL <https://arxiv.org/abs/2503.04721>.
- [37] Guan-Ting Lin, Shih-Yun Shan Kuan, Jiatong Shi, Kai-Wei Chang, Siddhant Arora, Shinji Watanabe, and Hung-yi Lee. Full-duplex-bench-v2: A multi-turn evaluation framework for duplex dialogue systems with an automated examiner. In *Proceedings of ACL*, 2026. URL <https://aclanthology.org/2026.acl-short.4/>.
- [38] Tanya Stivers, N. J. Enfield, Penelope Brown, Christina Englert, Makoto Hayashi, Trine Heinemann, Gertie Hoymann, Federico Rossano, Jan Peter de Ruiter, Kyung-Eun Yoon, and Stephen C. Levinson. Universals and cultural variation in turn-taking in conversation. *Proceedings of the National Academy of Sciences*, 106(26):10587–10592, 2009. doi: 10.1073/pnas.0903616106.